

Express CRUD

Cheat sheet + tutorial pratico su /posts

Obiettivo

Saper creare route Express con GET, POST, PUT e DELETE e collegarle ai controller index, show, store, update e destroy.

Lezioni riassunte

CRUD 1 - Index, Show, Delete

Lettura e cancellazione, parametri URI, query string, status HTTP e controller.

CRUD 2 - Store, Update

Creazione e modifica, body JSON con `express.json()` e dati in memoria.

Route	Controller	Risultato
GET /posts	index	Legge tutti i post
GET /posts/:id	show	Legge un post
POST /posts	store	Crea un post
PUT /posts/:id	update	Modifica un post
DELETE /posts/:id	destroy	Elimina un post

Struttura usata

Restiamo vicini alla repo: `app.js`, `routers/posts.js`, `data/posts.js`. Aggiungiamo solo `controllers/posts.js`.

1. CRUD e request: cheat sheet

Operazione	Metodo + URI	Controller	Status
Read all	GET /posts	index	200
Read one	GET /posts/:id	show	200 / 404
Create	POST /posts	store	201 / 400
Update	PUT /posts/:id	update	200 / 400 / 404
Delete	DELETE /posts/:id	destroy	204 / 404

Dove leggiamo i dati?

Fonte	Esempio	Express
Path parameter	/posts/3	req.params.id
Query string	/posts?tag=dolci	req.query.tag
JSON body	{"title": "..."}	req.body

Regola pratica

params identifica spesso una risorsa; query filtra o modifica la richiesta; body contiene i dati da creare o aggiornare.

app.js

La repo monta già il router su /posts. Per POST e PUT aggiungiamo `express.json()` prima del router.

```
import express from "express";
import postsRouter from "../routers/posts.js";

const app = express();
const port = 3000;

app.use(express.json());
app.use("/posts", postsRouter);

app.use(express.static("public"));

app.get("/", (req, res) => {
  res.send("Server del mio blog");
});

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`);
});
```

Perché `express.json()` prima delle route?

Perché deve leggere il JSON e creare `req.body` prima che i controller di POST e PUT vengano eseguiti.

2. Index e Show - GET

Nel file controllers/posts.js importiamo l'array già presente in data/posts.js.

index - GET /posts

```
import posts from "../data/posts.js";

export const index = (req, res) => {
  const { tag } = req.query;

  if (!tag) {
    return res.status(200).json(posts);
  }

  const filteredPosts = posts.filter((post) =>
    post.tags.includes(tag)
  );

  return res.status(200).json(filteredPosts);
};
```

- GET /posts → tutti i post
- GET /posts?tag=dolci → solo i post con quel tag

show - GET /posts/:id

```
export const show = (req, res) => {
  const id = Number(req.params.id);
  const post = posts.find((post) => post.id === id);

  if (!post) {
    return res.status(404).json({
      message: "Post non trovato",
    });
  }

  return res.status(200).json(post);
};
```

Da ricordare

req.params.id arriva come stringa. Con gli id numerici della repo usiamo Number(req.params.id).

3. Store - POST /posts

Store legge il body, crea un nuovo oggetto e lo aggiunge all'array con push().

Body di esempio

```
{
  "title": "Scialatielli ai frutti di mare",
  "content": "Un primo piatto della Costiera Amalfitana.",
  "image": "/images/scialatielli.jpeg",
  "tags": ["pasta", "pesce", "primi"]
}
```

Controller store

```
export const store = (req, res) => {
  const { title, content, image, tags } = req.body;

  if (!title || !content) {
    return res.status(400).json({
      message: "Title e content sono obbligatori",
    });
  }

  const newPost = {
    id: posts.length
      ? Math.max(...posts.map((post) => post.id)) + 1
      : 1,
    title,
    content,
    image,
    tags: tags ?? [],
  };

  posts.push(newPost);

  return res
    .status(201)
    .location(`/posts/${newPost.id}`)
    .json(newPost);
};
```

Flusso

1) leggi req.body; 2) valida i campi essenziali; 3) crea l'id; 4) fai posts.push(newPost); 5) rispondi con 201 Created.

Perché Location?

L'header Location indica l'URI della risorsa appena creata. Restituire anche il nuovo post nel body è comodo per il client.

4. Update e Destroy

update - PUT /posts/:id

```
export const update = (req, res) => {
  const id = Number(req.params.id);
  const post = posts.find((post) => post.id === id);

  if (!post) {
    return res.status(404).json({
      message: "Post non trovato",
    });
  }

  const { title, content, image, tags } = req.body;

  if (!title || !content) {
    return res.status(400).json({
      message: "Title e content sono obbligatori",
    });
  }

  post.title = title;
  post.content = content;
  post.image = image;
  post.tags = tags ?? [];

  return res.status(200).json(post);
};
```

PUT in questo tutorial

Lo trattiamo come aggiornamento completo dei campi modificabili. L'id arriva dall'URI e non cambia.

destroy - DELETE /posts/:id

```
export const destroy = (req, res) => {
  const id = Number(req.params.id);
  const index = posts.findIndex((post) => post.id === id);

  if (index === -1) {
    return res.status(404).json({
      message: "Post non trovato",
    });
  }

  posts.splice(index, 1);

  return res.status(204).send();
};
```

find() vs findIndex()

find() restituisce l'oggetto; findIndex() la posizione. Per splice() ci serve la posizione. Se non trova nulla, findIndex() restituisce -1.

5. controllers/posts.js completo

Questo è l'unico file nuovo: contiene tutta la logica CRUD.

```
import posts from "../data/posts.js";

export const index = (req, res) => {
  const { tag } = req.query;
  if (!tag) return res.status(200).json(posts);

  const filteredPosts = posts.filter((post) =>
    post.tags.includes(tag)
  );
  return res.status(200).json(filteredPosts);
};

export const show = (req, res) => {
  const id = Number(req.params.id);
  const post = posts.find((post) => post.id === id);

  if (!post) {
    return res.status(404).json({ message: "Post non trovato" });
  }
  return res.status(200).json(post);
};

export const store = (req, res) => {
  const { title, content, image, tags } = req.body;

  if (!title || !content) {
    return res.status(400).json({
      message: "Title e content sono obbligatori",
    });
  }

  const newPost = {
    id: posts.length
      ? Math.max(...posts.map((post) => post.id)) + 1
      : 1,
    title,
    content,
    image,
    tags: tags ?? [],
  };

  posts.push(newPost);

  return res.status(201)
    .location(`/posts/${newPost.id}`)
    .json(newPost);
};
```

5. controllers/posts.js completo - continua

```
export const update = (req, res) => {
  const id = Number(req.params.id);
  const post = posts.find((post) => post.id === id);

  if (!post) {
    return res.status(404).json({ message: "Post non trovato" });
  }

  const { title, content, image, tags } = req.body;

  if (!title || !content) {
    return res.status(400).json({
      message: "Title e content sono obbligatori",
    });
  }

  post.title = title;
  post.content = content;
  post.image = image;
  post.tags = tags ?? [];

  return res.status(200).json(post);
};

export const destroy = (req, res) => {
  const id = Number(req.params.id);
  const index = posts.findIndex((post) => post.id === id);

  if (index === -1) {
    return res.status(404).json({ message: "Post non trovato" });
  }

  posts.splice(index, 1);
  return res.status(204).send();
};
```

Persistenza in memoria

Le modifiche restano finché il processo Node è acceso. Al riavvio, l'array torna ai valori scritti in data/posts.js.

6. routers/posts.js completo

Il router diventa molto semplice: collega ogni route al suo controller.

```
import express from "express";

import {
  destroy,
  index,
  show,
  store,
  update,
} from "../controllers/posts.js";

const router = express.Router();

router.get("/", index);
router.get("/:id", show);
router.post("/", store);
router.put("/:id", update);
router.delete("/:id", destroy);

export default router;
```

Come leggerlo

`router.get('/', index)` significa: quando arriva GET /posts, Express esegue index.

Flusso di una request

Passo	Parte	Esempio
1	Client	PUT /posts/2 + JSON body
2	app.js	<code>express.json()</code> legge il body
3	Router	<code>router.put("/:id", update)</code>
4	Controller	update legge params/body e modifica posts
5	Response	status + JSON al client

7. Prova le route e ripassa

Con REST Client di VS Code puoi creare un file test.http:

```
@baseUrl = http://localhost:3000

### INDEX
GET {{baseUrl}}/posts

### QUERY STRING
GET {{baseUrl}}/posts?tag=dolci

### SHOW
GET {{baseUrl}}/posts/1

### STORE
POST {{baseUrl}}/posts
Content-Type: application/json

{
  "title": "Nuovo post",
  "content": "Contenuto del nuovo post",
  "image": "/images/nuovo.jpeg",
  "tags": ["test"]
}

### UPDATE
PUT {{baseUrl}}/posts/1
Content-Type: application/json

{
  "title": "Titolo aggiornato",
  "content": "Contenuto aggiornato",
  "image": "/images/ciambellone.jpeg",
  "tags": ["dolci", "aggiornato"]
}

### DESTROY
DELETE {{baseUrl}}/posts/1
```

Checklist finale

- `express.json()` è prima delle route.
- `req.params` per `:id`.
- `req.query` per filtri facoltativi.
- `req.body` per POST e PUT.
- Show, update e destroy gestiscono il caso 404.
- Store risponde 201; destroy risponde 204.
- Il router decide quale controller eseguire; il controller contiene la logica.

Domanda	Risposta breve
Cos'è index?	Restituisce la collezione di risorse.
Cos'è show?	Restituisce una singola risorsa tramite id.
Cos'è store?	Crea e aggiunge una nuova risorsa.
Cos'è update?	Modifica una risorsa esistente.
Cos'è destroy?	Elimina una risorsa.
Perché <code>express.json()</code> ?	Per leggere il JSON del body in <code>req.body</code> .